Supplementary Material to the manuscript

A Closed-Loop Cardiovascular Model as a Teaching Framework

for Physics, Mathematical Modeling, and Scientific Computing

From Equations to Guided Experiments: A Modular Python Framework for Teaching Closed-Loop Cardiovascular Dynamics

M. Varanis*

How to read this supplement

This supplement was designed to serve three distinct readers. Pick the route that matches your goal.

- **R** **A reviewer or researcher who wants to reproduce the paper.** Go directly to §4 *(Simulation workflow)*, §5 *(Reproducibility instructions)*, and §10 *(Repository and availability)*.
- **T** **An instructor planning to use this in class.** Focus on §7 *(Suggested computational experiments)* and §9 *(Educational use)*. Each experiment is self-contained and can be assigned as a lab.
- **E** **C** **A student who wants to understand the bridge between the equations of the paper and the Python code.** Read §3 *(Mapping between equations and implementation)* and §6 *(Representative code excerpts)*.

The purpose of this document is not to reproduce large blocks of raw source code, but to mediate between the mathematical model presented in the main text and its computational realization in Python.

Abbreviations. The main physiological and hemodynamic abbreviations used below are: LV = left ventricle, RV = right ventricle, PV = pressure-volume, MAP = mean arterial pressure, SV = stroke volume, CO = cardiac output, EF = ejection fraction, EDV = end-diastolic volume, and ESV = end-systolic volume. Other abbreviations follow the main manuscript.

*Nonlinear Dynamics and Signal Processing Group, Institute of Physics, Federal University of Mato Grosso do Sul, Brazil. marcus.varanis@ufms.br varanis.cc

Contents

1 Purpose and scope **3**

2 Code organization **3**
2.1 Physiological observables and cycle-derived metrics 4

3 Mapping between equations and implementation **6**

4 Simulation workflow **7**

5 Reproducibility instructions **8**

6 Representative code excerpts **9**
6.1 The right-hand side `rhs(t, y)` 9
6.2 The cardiac activation function $a(t)$ 10
6.3 Derivation of intracavitary ventricular pressures 10
6.4 A parameter-variation study in three lines 10

7 Suggested computational experiments **11**

8 Technical protocol for local sensitivity and practical-identifiability screening **14**

9 Educational use **17**

10 Repository and availability **20**

1 Purpose and scope

This supplement accompanies a teaching-oriented reformulation of a classical closed-loop lumped-parameter cardiovascular model (see the main manuscript for the physical and mathematical derivation). Its purpose is not to extend the physiological scope of the paper, but to enhance *access*: the supplement transforms the model from a set of equations printed in a journal into a computational environment that a colleague can install, run, modify, and teach from in an afternoon.

Four functions are pursued simultaneously.

1. **Reproducibility** - every number and every figure in the main manuscript must be regenerable from the published code with a fixed set of commands (§5).
2. **Computational clarity** - the code is refactored into a modular, dataclass-based architecture with a strict separation between model, integrator, post-processing, and visualization (§2).
3. **Pedagogical design** - the supplement offers a set of six guided experiments, each taking the form *question* → *one-line code change* → *expected figure* → *physiological interpretation* (§7).
4. **Explicit bridging between equations and code** — every equation of the main manuscript is mapped to a specific function, method, or line of the package (§3).

What this supplement is *not*: a verbatim dump of source files, a catalog of classes, or a stack of code pages presented without mediation. Those pages will be released in the public repository described in §10; what appears below is the mediation.

2 Code organization

The package `cardiosim` is organized around a single conceptual pipeline, shown in Fig. 1: the user assembles a simulation from immutable parameters and an initial state, runs it through an adaptive ODE solver, derives physical observables, and renders figures with the group's visual identity.

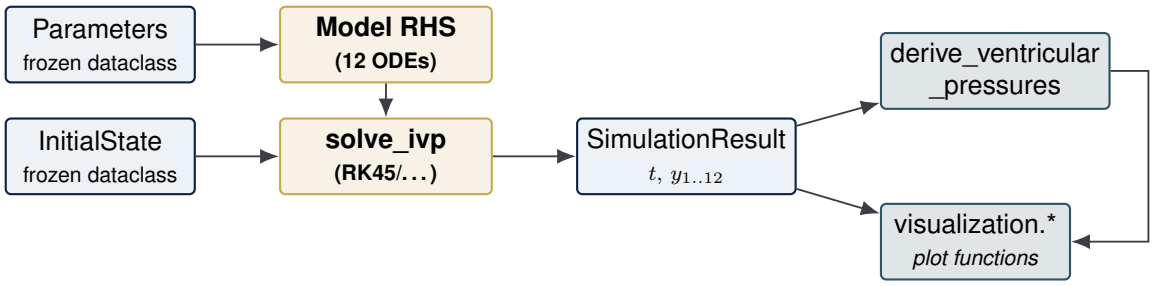


Figure 1: Data flow of the `cardiosim` package. Blue boxes are self-describing data objects; gold boxes are computational steps; teal boxes are derived observables and visual output. No state persists outside the central `SimulationResult`.

The file layout mirrors the conceptual pipeline:

Module	Role	Corresponds to
`parameters.py`	Physiological constants	Table of parameters
`initial_state.py`	Initial $\mathbf{x}(0)$	Table 1 of manuscript
`model.py`	Right-hand side $\Phi(\mathbf{x}, t)$ and valve laws	Eqs. (5)–(17)
`simulate.py`	Adaptive integration wrapper	SciPy `solve_ivp`
`postprocess.py`	Derived $P_{LV}(t)$, $P_{RV}(t)$, cycle metrics	Eqs. (11), (15), Table 2
`visualization.py`	Pure plotting functions	Figs. 6–14
`style.py`	House visual identity	Navy / gold palette

T Pedagogical note — The separation of concerns enforced in this layout is the central design choice of the refactor. Students can isolate *which* module they want to modify to answer a given question: changing a parameter never requires touching the solver; changing the solver never requires touching the model; adding a figure never requires re-running a simulation.

2.1 Physiological observables and cycle-derived metrics

The state-space solution provides time-dependent pressures, flow rates, and ventricular volumes. Several quantities commonly used in cardiovascular interpretation, however, are not independent state variables but cycle-derived metrics obtained by post-processing the converged periodic solution. Distinguishing among simulated states, prescribed inputs, and derived observables is therefore essential for connecting the mathematical formulation to its physiological interpretation.

The pressure states represent distinct locations within the closed circulation. The aortic pressure is $P_{ao}(t) = x_1(t)$, whereas $x_3(t)$ represents the distal systemic arterial pressure. On the pulmonary side, $x_7(t)$ and $x_9(t)$ denote the proximal and distal pulmonary arterial pressures, respectively. The states $x_5(t)$ and $x_{11}(t)$ represent the right- and left-atrial filling pressures. By contrast, the intracavitary ventricular pressures $P_{vR}(t)$ and $P_{vL}(t)$ are derived from the corresponding pressure-source, elastance, volume, and myocardial-resistance terms.

Let $I_k = [t_k, t_k + t_c]$ denote a complete cardiac cycle selected after convergence to the periodic regime. The systolic and diastolic aortic pressures are defined as the maximum and minimum aortic pressures attained during that cycle:

$$P_{\mathrm{sys}} = \max_{t \in I_k} P_{ao}(t),$$
$$P_{\mathrm{dia}} = \min_{t \in I_k} P_{ao}(t).$$

Their difference defines the aortic pulse pressure,

$$PP = P_{\mathrm{sys}} - P_{\mathrm{dia}},$$

which measures the amplitude of the aortic pressure excursion over one cardiac cycle. The exact cycle-averaged aortic pressure is

$$\overline{P}_{ao} = \frac{1}{t_c} \int_{t_k}^{t_k + t_c} P_{ao}(t)\, dt.$$

For comparison with the conventional resting approximation, mean arterial pressure may also be estimated from the systolic and diastolic levels as

$$\mathrm{MAP}_{\mathrm{est}} = P_{\mathrm{dia}} + \frac{1}{3}\left(P_{\mathrm{sys}} - P_{\mathrm{dia}}\right).$$

The latter expression is a waveform-based approximation and should be distinguished from the exact temporal average $\overline{P}_{ao}$ calculated from the simulated pressure trace.

Heart rate is a prescribed input rather than an emergent output of the present model. It is determined by the cardiac period according to

$$HR = \frac{60}{t_c},$$

where t_c is expressed in seconds and HR in beats per minute. Consequently, the model does not simulate chronotropic regulation: changing the heart rate requires changing the prescribed cardiac period.

Under the convention $V_{lp,L} = V_{lp,R} = 0$, the states $x_{12}(t)$ and $x_6(t)$ are the absolute left- and right-ventricular volumes, respectively. Because $1\ \mathrm{cm}^3 = 1\ \mathrm{mL}$, their numerical values may be reported directly in milliliters. For the left ventricle, the end-diastolic and end-systolic volumes are extracted from a converged cycle as

$$EDV_{\mathrm{LV}} = \max_{t \in I_k} x_{12}(t),$$
$$ESV_{\mathrm{LV}} = \min_{t \in I_k} x_{12}(t).$$

The corresponding stroke volume and ejection fraction are

$$SV_{\mathrm{LV}} = EDV_{\mathrm{LV}} - ESV_{\mathrm{LV}},$$
$$EF_{\mathrm{LV}} = 100\,\frac{SV_{\mathrm{LV}}}{EDV_{\mathrm{LV}}}.$$

Here, SV_{LV} is the gross ventricular volume excursion per beat, whereas EF_{LV} is the percentage of the end-diastolic volume expelled during that excursion. The ejection fraction is loading-dependent and should not be interpreted as an isolated measure of myocardial contractility.

For a competent aortic valve, the cardiac output calculated from the left-ventricular stroke volume is

$$CO_{\mathrm{gross}} = \frac{SV_{\mathrm{LV}}\,HR}{1000},$$

with CO_{gross} expressed in $\mathrm{L\,min^{-1}}$. When aortic regurgitation is permitted, however, the ventricular volume excursion is not identical to the volume delivered to the systemic circulation. The forward and reverse aortic volumes are then obtained from the signed aortic-valve flow $Z_{oL}(t)$:

$$V_{\mathrm{fwd}} = \int_{I_k} \max[Z_{oL}(t), 0]\ dt,$$
$$V_{\mathrm{rev}} = -\int_{I_k} \min[Z_{oL}(t), 0]\ dt.$$

The net volume delivered to the aorta, the regurgitant fraction, and the effective cardiac output are consequently

$$V_{\mathrm{net}} = V_{\mathrm{fwd}} - V_{\mathrm{rev}},$$
$$RF = 100\,\frac{V_{\mathrm{rev}}}{V_{\mathrm{fwd}}},$$
$$CO_{\mathrm{eff}} = \frac{V_{\mathrm{net}}\,HR}{1000}.$$

This distinction is essential in the modified aortic-valve experiment: the coefficient Ref specifies the ratio of reverse to forward valve conductance and is therefore an input parameter, whereas RF is a cycle-integrated simulation output.

Within the present model, end-diastolic volume and atrial filling pressure provide operational indicators of preload, while systemic resistance contributes to the load opposed to left-ventricular ejection. The pressure–volume loop combines these loading conditions with ventricular dynamics. Its enclosed area,

$$W_{\mathrm{stroke}} = \left| \oint P_v\, dV \right|,$$

represents the magnitude of the mechanical work performed by the ventricle during one cardiac cycle. Thus, the model supports a clear hierarchy of interpretation: the state trajectory describes the instantaneous dynamical condition, cycle-derived metrics summarize its periodic behavior, and physiological observables connect those numerical quantities to cardiovascular function.

3 Mapping between equations and implementation

This is the bridge that turns the manuscript from a set of formulas into an executable model. Each row of Table 1 points to the specific place where the equation is evaluated at run time. For compactness, `CLC` denotes the `ClosedLoopCardiovascular` class.

Table 1: Mapping between the equations of the main manuscript and their implementation in `cardiosim`. All equation numbers refer to the main text.

Equation / quantity	State variable or symbol	Code location
Constitutive relations (RLC analogy)		
Ohm's law, Eq. (1)	$v = R\,i$	Implicit in `rhs`: terms $R_i x_j$
Capacitor, Eq. (3)	$i = C\,\dot{v}$	Implicit in `rhs`: divisions by C_i
Inductor, Eq. (4)	$v = L\,\dot{i}$	Implicit in `rhs`: divisions by L_i
Ventricular subsystem		
Activation, Eq. (5)	$a(t)$	`CLC.activation`
Systolic duration	$t_s = 0.16 + 0.3\,t_c$	`Parameters.systolic_duration`
Right elastance, Eq. (7)	$E_R(t) = E_{d,R} + E_{s,R}\,a(t)$	`CLC.elastance_right`
Left elastance, Eq. (7)	$E_L(t) = E_{d,L} + E_{s,L}\,a(t)$	`CLC.elastance_left`
Right source, Eq. (7)	$P_R(t) = P_{0R}\,a(t)$	`CLC.source_pressure_right`
Left source, Eq. (7)	$P_L(t) = P_{0L}\,a(t)$	`CLC.source_pressure_left`
Valve logic (diodes)		
Tricuspid switch, Eq. (8)	$S_2 = \mathbf{1}_{\Delta P_2 > 0}$	`s2` in `CLC.rhs`
Pulmonary switch, Eq. (8)	$S_3 = \mathbf{1}_{\Delta P_3 > 0}$	`s3` in `CLC.rhs`
Mitral switch, Eq. (12)	$S_4 = \mathbf{1}_{\Delta P_4 > 0}$	`s4` in `CLC.rhs`
Aortic conductance factor, Eq. (12)	$S_1 = 1$ for $\Delta P_1 \geq 0$; $S_1 = \mathrm{Ref}$ otherwise	`aortic_valve_factor` in `model.py`
Signed aortic-valve flow	$Z_{oL} = S_1 \Delta P_1 / (R_L + R_1)$	`aortic_valve_flow` in `model.py`
Valve pressure differences		
ΔP_1, Eq. (13)	$P_L + x_{12} E_L - x_1$	`dP1` in `CLC.rhs`
ΔP_2, Eq. (9)	$x_5 - P_R - x_6 E_R$	`dP2` in `CLC.rhs`
ΔP_3, Eq. (9)	$P_R + x_6 E_R - x_7$	`dP3` in `CLC.rhs`
ΔP_4, Eq. (13)	$x_{11} - P_L - x_{12} E_L$	`dP4` in `CLC.rhs`
Intracavitary ventricular pressures		
P_{vR}, Eq. (11)	$P_R + E_R x_6 - R_R Z_{oR}$	`derive_ventricular_pressures`
P_{vL}, Eq. (15)	$P_L + E_L x_{12} - R_L Z_{oL}$	`derive_ventricular_pressures`
Ventricular-volume convention	$V_{lp,R} = V_{lp,L} = 0$; $x_6 = V_{RV}$, $x_{12} = V_{LV}$	`initial_state.py` and `postprocess.py`
State-space formulation		
$\dot{\mathbf{x}} = \mathbf{\Phi}(\mathbf{x}, t)$, Eq. (16)	12-dim vector field	`CLC.rhs`
Twelve coupled ODEs, Eq. (17)	$\dot{x}_1, \ldots, \dot{x}_{12}$	Lines `dx1`–`dx12` in `rhs`

Equation / quantity	State variable or symbol	Code location
Post-processed observables		
Stroke volume	$SV = \mathrm{EDV} - \mathrm{ESV}$	`cycle_metrics`
Cardiac output	$CO = SV \cdot HR/1000$ (gross LV output)	`cycle_metrics`
Signed aortic-flow metrics	forward, reverse, and net cycle volumes; effective CO	`aortic_flow_metrics`
Closed-loop volume invariant	$\sum_i C_i P_i + V_{RV} + V_{LV}$	`total_blood_volume`
Mean arterial pressure	$MAP \approx P_d + \frac{1}{3}(P_s - P_d)$	`cycle_metrics`
Ejection fraction	$EF = 100\,SV/\mathrm{EDV}$	`cycle_metrics`

The sign convention is LV-to-aorta positive. The parameter $0 \le \mathrm{Ref} < 1$ is the reverse-to-forward conductance ratio. Therefore, when $\Delta P_1 < 0$ and $\mathrm{Ref} > 0$, the factor S_1 remains nonnegative and $Z_{oL} < 0$, which transfers volume from the aortic compartment back to the left ventricle. The baseline $\mathrm{Ref} = 0$ recovers an ideal non-return valve.

E C From equation to code

A word on the intracavitary pressure. In earlier implementations, the pressure–volume loops were drawn using only the active source $P_0\,a(t)$. That quantity is *not* the intracavitary ventricular pressure: it omits the elastance–volume contribution, and when the outflow valve is open, the myocardial viscosity drop.
Following Eqs. (11) and (15) of the main text, the correct expression is implemented in a single place, `postprocess.derive_ventricular_pressures`, and is reused by every visualization that shows $P_{LV}(t)$, $P_{RV}(t)$, or the pressure–volume loops. This eliminates the risk of inconsistency between time-series plots and PV diagrams.
The ventricular-volume states are interpreted with $V_{lp,R} = V_{lp,L} = 0$. They are therefore absolute chamber volumes, not volume deviations. The same convention is used when computing EDV, ESV, stroke volume, ejection fraction, and the PV loops.

4 Simulation workflow

The workflow from “Python interpreter prompt” to “publication-ready figure” is four lines of code.

```
from cardiosim import (baseline_parameters, baseline_initial_state,
                       run_simulation, cycle_metrics)

result  = run_simulation(baseline_parameters(),
                         baseline_initial_state(), t_end=10.0)
metrics = cycle_metrics(result)
print(f"MAP = {metrics.MAP:.2f} mmHg, EF = {metrics.EF_LV:.2f}%")
```

Listing 1: Minimal end-to-end pipeline.

In pseudo-code, the chain of responsibility is:

1. `Parameters` and `InitialState` are assembled as frozen dataclasses.
2. `ClosedLoopCardiovascular(parameters, initial_state)` exposes `rhs(t, y)`, a pure function of the state.
3. `solve_ivp` integrates `rhs` from t_0 to t_{end} with adaptive step and tight tolerances.

4. The output is wrapped into a `SimulationResult` that carries both the trajectory and the provenance (parameters, initial state, solver, tolerances, wall time, `nfev`).
5. `derive_ventricular_pressures` and `cycle_metrics` compute observables without re-running the simulation.
6. `visualization.*` functions consume `SimulationResult` and return `Figure` instances from `matplotlib` — they do not save to disk themselves, so the caller stays in control.

Default integration settings

The simulations use the SciPy RK45 adaptive Runge–Kutta pair with `rtol = 1e-10` and `atol = 1e-12`. The long-horizon stability analysis in the manuscript uses $180\,\mathrm{s}$, corresponding to 225 cardiac cycles at the baseline period $t_c = 0.8\,\mathrm{s}$. Figures display selected time windows, whereas cycle metrics are extracted only after the transient has decayed. Shorter 10–$20\,\mathrm{s}$ runs are retained in the guided laboratory examples because they reproduce the stabilized baseline observables at substantially lower classroom cost. A cross-solver benchmark is reported in Table 2 and discussed in Experiment 4 (§7).

5 Reproducibility instructions

R Reproducibility check — Every figure in the main manuscript is intended to be regenerable from the published code using a fixed set of commands. Because the model is deterministic, repeated runs in the same software environment are expected to produce numerically equivalent results up to floating-point roundoff.
At the present pre-acceptance stage, repository and archival details are described provisionally in §10. Upon acceptance of the manuscript, the final public release will include the version-controlled source code, the example scripts used to regenerate the paper figures, and a citable archived snapshot with persistent DOI.

Installation

The public installation workflow will be released upon acceptance of the manuscript, together with the version-controlled repository and the archived snapshot of the package. The codebase requires Python ≥ 3.10 and depends on the scientific Python stack listed below.

Software dependencies

The minimum supported versions are listed in `requirements.txt`; the constraints are reproduced here:

Package	Version
`python`	$\geq$ 3.10 (3.12 used for validation)
`numpy`	$\geq$ 1.24
`scipy`	$\geq$ 1.11
`matplotlib`	$\geq$ 3.7
`pytest`	$\geq$ 7.0 (tests only)

Regenerating the figures

Upon acceptance, the public release will include dedicated example scripts for regenerating the baseline results, the parameter-variation studies, and the manuscript figures. Output will be written to `paper_figures/` in both raster (PNG) and vector (PDF) formats.

Verification tests

The package includes 13 automated tests that verify the main structural, physical, and numerical properties of the implementation. They check (i) shape and finiteness of the RHS; (ii) forward, closed-valve, and regurgitant directions of the signed aortic flow; (iii) admissible parameter ranges; (iv) algebraic and trajectory-level conservation of closed-loop blood volume; (v) completion and periodicity of the baseline run; (vi) hemodynamic metrics and numerical regression against the reported baseline; (vii) the convention $V_{lp,R} = V_{lp,L} = 0$; and (viii) the topology of the pressure–volume loop. The regurgitant branch is exercised both as a constitutive-law unit test and through a complete pathological simulation.

6 Representative code excerpts

Four excerpts are reproduced below, chosen because they mediate the transition from equations to implementation. The full source will be released in the repository described in §10; the goal here is to make the reader confident that “the code looks like the paper”.

6.1 The right-hand side `rhs(t, y)`

This is the direct image of Eq. (17) of the manuscript. Variable names follow Table 1. The tricuspid, pulmonary, and mitral valves use ideal-diode switches. The aortic branch uses a signed constitutive law, with positive flow directed from the left ventricle to the aorta and negative flow directed from the aorta to the ventricle.

```
def aortic_valve_factor(self, pressure_difference):
    dP = np.asarray(pressure_difference)
    return np.where(dP >= 0.0, 1.0, self.p.aortic_reflux)

def aortic_valve_flow(self, pressure_difference):
    dP = np.asarray(pressure_difference)
    resistance = self.p.R_myo_left + self.p.R_aortic_valve
    return self.aortic_valve_factor(dP) * dP / resistance

def rhs(self, t, y):
    p = self.p
    (x1, x2, x3, x4, x5, x6, x7, x8, x9, x10, x11, x12) = y

    P_L = self.source_pressure_left(t)
    P_R = self.source_pressure_right(t)
    E_L = self.elastance_left(t)
    E_R = self.elastance_right(t)

    dP1 = P_L + x12 * E_L - x1          # aortic    (LV -> aorta)
    dP2 = x5 - P_R - x6 * E_R           # tricuspid (RA -> RV)
    dP3 = P_R + x6 * E_R - x7           # pulmonary (RV -> PA)
    dP4 = x11 - P_L - x12 * E_L         # mitral    (LA -> LV)

    s2 = 1.0 if dP2 > 0.0 else 0.0
    s3 = 1.0 if dP3 > 0.0 else 0.0
    s4 = 1.0 if dP4 > 0.0 else 0.0
```

```

    Z_ao_out  = self.aortic_valve_flow(dP1)
    Z_tri_in  = (s2 * dP2) /  p.R_tricuspid
    Z_pul_out = (s3 * dP3) / (p.R_myo_right + p.R_pulmonary_valve)
    Z_mit_in  = (s4 * dP4) /  p.R_mitral

    # Twelve coupled ODEs follow, one per physiological compartment.
```

Listing 2: Core of `model.py` — the vector field $\mathbf{\Phi}(\mathbf{x}, t)$.

6.2 The cardiac activation function $a(t)$

Equation (5) of the manuscript, expressed in one line of NumPy. The vectorised form means this function can be evaluated on the entire solution grid at once, which is useful for post-processing.

```
def activation(self, t):
    t_c = self.p.cardiac_period
    t_s = self.p.systolic_duration      # 0.16 + 0.3 t_c
    t_m = np.mod(t, t_c)
    return np.where(t_m < t_s,
                    0.5 * (1.0 - np.cos(2.0 * np.pi * t_m / t_s)),
                    0.0)
```

Listing 3: The activation function $a(t)$, Eq. (5).

6.3 Derivation of intracavitary ventricular pressures

This is where the PV-loop bug of earlier implementations is fixed at the root. The function is the single source of truth for $P_{LV}(t)$ and $P_{RV}(t)$ in all downstream plots.

```
def derive_ventricular_pressures(result, *, subtract_myocardial_drop=True):
    t, m, p = result.t, result.model, result.parameters

    a    = m.activation(t)
    E_L  = p.E_d_left  + p.E_s_left  * a
    E_R  = p.E_d_right + p.E_s_right * a

    P_LV = p.P0_left  * a + E_L * result.V_left_vent
    P_RV = p.P0_right * a + E_R * result.V_right_vent

    if subtract_myocardial_drop:
        dP1 = P_LV - result.P_aortic
        dP3 = P_RV - result.P_pulmonary_arterial
        s3  = (dP3 > 0).astype(float)
        Z_oL = m.aortic_valve_flow(dP1)
        Z_oR = (s3 * dP3) / (p.R_myo_right + p.R_pulmonary_valve)
        P_LV = P_LV - p.R_myo_left  * Z_oL
        P_RV = P_RV - p.R_myo_right * Z_oR

    return VentricularPressures(P_LV=P_LV, P_RV=P_RV)
```

Listing 4: Intracavitary pressure derivation (Eqs. 11 and 15).

6.4 A parameter-variation study in three lines

```
p_hyper = baseline_parameters().modified(R_sys=2.0 * baseline_parameters().
    R_sys)
result_hyper = run_simulation(p_hyper, baseline_initial_state(), t_end=15.0)
plot_parameter_variation(result_baseline, result_hyper,
```

```
                                perturbed_label="R_sys x 2.0").savefig("hyper.pdf")
```

Listing 5: Setting up a hypertensive scenario with doubled systemic resistance.

T Pedagogical note — `.modified(...)` returns a new `Parameters` instance rather than mutating the original. This is what makes parameter-variation sweeps safe and deterministic: the baseline `Parameters` object can be passed to any number of perturbed runs without ever being altered.

7 Suggested computational experiments

The six experiments below are designed to be run in the order they are presented, or selected individually as self-contained laboratory activities. Each follows the same template:

- **X Question** — what are we asking?
- **C Setup** — one- or two-line code change.
- **E Expected outcome** — what should the figure or metric show?
- **T Discussion** — how should the result be interpreted physically and computationally?

Suggested total time in a 3-hour lab: one experiment per ∼25 minutes, with 30 minutes of baseline orientation at the start.

Run the following common setup once before the selected activities:

```
from cardiosim import (
    baseline_parameters, baseline_initial_state, run_simulation,
    cycle_metrics, aortic_flow_metrics,
)
from cardiosim.visualization import (
    plot_parameter_variation, plot_pv_loops,
)

p0 = baseline_parameters()
x0 = baseline_initial_state()
result_baseline = run_simulation(p0, x0, t_end=15.0)
```

X Experiment 1. Sensitivity to systemic resistance

X Question. How does mean arterial pressure (MAP) respond as the systemic peripheral resistance R_{sys} varies from half of its baseline value to twice its baseline value?

C Setup.

```
import numpy as np

factors = np.linspace(0.5, 2.0, 10)
maps = []

for f in factors:
    p = baseline_parameters().modified(
        R_sys=f * baseline_parameters().R_sys
    )
    r = run_simulation(p, baseline_initial_state(), t_end=20.0)
    maps.append(cycle_metrics(r).MAP)
```

E Expected outcome. A roughly increasing relation between MAP and R_{sys} over this range, with stroke volume decreasing and ejection fraction slightly reduced. Plot MAP against the resistance factor.

T Discussion. The experiment is the numerical analogue of the afterload concept: in-

creased downstream resistance raises arterial pressure while simultaneously increasing the mechanical load on the left ventricle, thereby reducing pump efficiency.

X Experiment 2. Contractility and ejection fraction

X Question. If the left-ventricular systolic elastance $E_{s,L}$ is reduced by 50% (a crude model of systolic dysfunction), what happens to the ejection fraction and to the pressure–volume loop?

C Setup.

```
p = baseline_parameters()
p_weak = p.modified(E_s_left=0.5 * p.E_s_left)

r_weak = run_simulation(p_weak, baseline_initial_state(), t_end=15.0)
print(cycle_metrics(r_weak))

plot_parameter_variation(
    result_baseline,
    r_weak,
    perturbed_label="0.5 * E_s,L"
).savefig("reduced_contractility.pdf")
```

E Expected outcome. The LV pressure–volume loop collapses vertically, the peak systolic pressure decreases, and the ejection fraction falls below the baseline ∼55%.

T Discussion. The simulation mimics the most elementary feature of reduced systolic function: the ventricle becomes mechanically weaker, ejects less blood per beat, and retains a larger residual volume. In a classroom setting, this opens a direct discussion of the role of E_s as a mechanical measure of ventricular contractility.

X Experiment 3. Signed aortic regurgitation and effective output

X Question. How does a finite reverse conductance alter the signed aortic flow, the left-ventricular pressure–volume loop, and the effective cardiac output?

C Setup.

```
p_regurg = p0.modified(
    aortic_reflux=0.05,
    R_aortic_valve=0.4 * p0.R_aortic_valve,
)
r_regurg = run_simulation(p_regurg, x0, t_end=15.0)
flow = aortic_flow_metrics(r_regurg)

print(f"reverse volume = {flow.regurgitant_volume:.2f} mL/cycle")
print(f"regurgitant fraction = {flow.regurgitant_fraction:.1f}%")
print(f"effective CO = {flow.effective_CO:.2f} L/min")

plot_parameter_variation(
    result_baseline, r_regurg, perturbed_label="Regurgitant"
).savefig("fig14_var_aortic_valve.pdf")
```

E Expected outcome. In the corrected simulation, the aortic-valve flow becomes negative whenever the aortic pressure exceeds the left-side driving pressure. For the illustrative parameters above, the last cycle gives a forward volume of approximately 75.68 mL, a reverse volume of 19.16 mL, a simulated regurgitant fraction of 25.31%, and an effective cardiac output of 4.24 L/min. The LV loop widens toward larger end-diastolic volumes as blood returns to the ventricle.

T Discussion. The input $\mathrm{Ref} = 0.05$ is the ratio of reverse to forward conductance, not a

clinical statement of “5% regurgitation.” The simulated regurgitant fraction is an output of the coupled dynamics and depends on both the conductance law and the time-varying pressure gradient. This distinction makes the experiment useful for teaching the difference between a model parameter and an emergent diagnostic quantity.

X Experiment 4. Comparison of ODE solvers

X Question. At the same tolerance, which adaptive integrator is most efficient for this problem, and how does efficiency change with tolerance?
C Setup. Run `examples/03_solver_benchmark.py`; then repeat with `rtol` swept in $\{10^{-6}, 10^{-8}, 10^{-10}\}$ and `atol` = `rtol` / 100.
E Expected outcome. For the smooth 0D dynamics considered here, explicit methods are typically competitive, whereas implicit methods may incur additional cost that is not required under baseline conditions. Compare wall time, number of RHS evaluations, and qualitative overlap of the resulting waveforms.
T Discussion. This is a natural bridge to a discussion of *stiffness*. Under baseline conditions, the model is not strongly stiff, so implicit methods may be unnecessarily expensive for a given target accuracy. Under more extreme parameter choices, however, the ranking may change. The exact ordering is problem-, tolerance-, and hardware-dependent, which is itself an important lesson in numerical analysis.

A reproducible benchmark on commodity hardware is shown below.

Table 2: Cross-solver benchmark on commodity hardware. Baseline run, $t_{\text{end}} = 10\,\text{s}$, `rtol` $= 10^{-10}$, `atol` $= 10^{-12}$. The exact ranking is implementation-, tolerance-, and hardware-dependent.

Solver	Steps	nfev	Wall time [s]	Status
RK45	17 905	113 852	3.06	success
DOP853	4 671	84 740	2.08	success
LSODA	20 000	66 287	1.70	success
Radau	43 222	315 216	15.22	success
BDF	27 839	62 095	4.92	success

X Experiment 5. Heart rate and cardiac output

X Question. How does cardiac output respond to changes in heart rate, with all other parameters held fixed?
C Setup.

```
hrs = []
cos = []
svs = []

for t_c in [0.5, 0.6, 0.8, 1.0, 1.2]:    # 120 to 50 bpm
    p = baseline_parameters().modified(cardiac_period=t_c)
    r = run_simulation(p, baseline_initial_state(), t_end=20.0)
    m = cycle_metrics(r)

    hrs.append(m.HR)
    cos.append(m.CO)
    svs.append(m.SV_LV)
```

```
    print(f"HR = {m.HR:.1f} bpm    CO = {m.CO:.2f} L/min    SV = {m.SV_LV:.1f
        } mL")
```

E Expected outcome. Cardiac output does *not* scale linearly with heart rate: at very high rates, reduced filling time shortens diastole and lowers stroke volume, so CO may saturate or even decrease. A plot of CO versus HR should make this trend visible.

T Discussion. The experiment operationalizes the distinction between *chronotropic* effects (rate) and *inotropic* effects (contractility). It is also a useful numerical sensitivity test because the temporal structure of the solution changes substantially, forcing the adaptive time-step controller to reorganize its sampling.

x Experiment 6. Long-time stability and periodic regime

X Question. Does the model preserve its limit-cycle geometry over a long simulation window?

C Setup.

```
import numpy as np

r_long = run_simulation(
    baseline_parameters(),
    baseline_initial_state(),
    t_end=300.0
)

def resampled_cycle(result, cycle_index, n=400):
    tc = result.parameters.cardiac_period
    start, stop = cycle_index * tc, (cycle_index + 1) * tc
    mask = (result.t >= start) & (result.t <= stop)
    tau = np.linspace(0.0, tc, n)
    values = np.interp(tau, result.t[mask] - start,
                       result.V_left_vent[mask])
    return values

early = resampled_cycle(r_long, cycle_index=10)
late = resampled_cycle(r_long, cycle_index=373)
drift = np.max(np.abs(late - early)) / np.ptp(late)

print(f"Relative cycle drift = {drift:.3e}")
```

E Expected outcome. Under tight tolerances, the relative drift between early and late stabilized cycles is expected to remain below 10^{-3}. The automated periodicity test uses a more conservative acceptance threshold of 5×10^{-3}.

T Discussion. In a methods-oriented course, this experiment connects directly to the notion of an attracting periodic orbit. Resistive elements dissipate mechanical energy, while the closed loop preserves the stored blood-volume invariant. The numerical question is whether integration also preserves the geometry of the stabilized limit cycle over long times. This opens a productive discussion about conservation laws, error accumulation, asymptotic dynamics, and the numerical representation of periodic behavior.

8 Technical protocol for local sensitivity and practical-identifiability screening

The parameter-variation studies above answer forward questions of the form “what changes when a selected coefficient is modified?” They do not constitute a formal sensitivity analysis and do not establish whether the model parameters can be inferred uniquely from a prescribed set of

observations. The protocol below introduces these two model- assessment problems in a compact, reproducible form. Its purpose is to quantify local output sensitivity and to diagnose poorly informed parameter combinations under explicitly defined observation maps.

Parameter subset and perturbation design

Consider the six-dimensional parameter subset

$$\mathbf{p}_s = [R_3,\, C_1,\, C_2,\, E_{s,L},\, R_7,\, C_5]^T\,, \tag{1}$$

selected to include resistive, compliant, and contractile mechanisms from both the systemic and pulmonary circuits. The corresponding code names and physical roles are listed in Table 3.

Table 3: Parameter subset used in the local sensitivity and practical-identifiability screening.

Symbol	Code name	Physical role
R_3	`R_sys`	Systemic peripheral resistance
C_1	`C_ao`	Aortic compliance
C_2	`C_sys_art`	Distal systemic arterial compliance
$E_{s,L}$	`E_s_left`	Left-ventricular systolic elastance
R_7	`R_pul`	Distal pulmonary vascular resistance
C_5	`C_pul_art`	Distal pulmonary vascular compliance

For each parameter p_j, construct symmetric perturbations

$$p_j^- = (1-\varepsilon)p_j, \qquad p_j^+ = (1+\varepsilon)p_j, \qquad \varepsilon = 0.05, \tag{2}$$

while all remaining parameters are held at their baseline values. This design requires twelve perturbed simulations in addition to the baseline run. The finite perturbation is intentionally small enough to probe the neighborhood of the reference configuration, but it should not be interpreted as an experimentally established uncertainty bound.

Convergence and phase standardization

The baseline configuration must first be integrated for 100 cardiac cycles. Its final state is then used as the initial condition for every perturbed case. Each perturbed configuration is integrated in blocks of ten complete cycles until

$$\eta_{\mathrm{cyc}} = \max_{1\le i\le 12} \frac{\left\| x_i^{(n)}(\phi) - x_i^{(n-1)}(\phi) \right\|_\infty}{\mathrm{ptp}\left(x_i^{(n)}(\phi) \right)} < 10^{-6}, \tag{3}$$

where the two final cycles are interpolated on the same phase grid. For a state with negligible peak-to-peak amplitude, the denominator is replaced by a finite scale based on its maximum absolute value. A case that does not satisfy Eq. (3) within 100 cycles must be reported as unconverged and must not be included in the sensitivity matrix.

After convergence, the final cycle of every simulation is sampled at $M = 400$ equally spaced phases, $\phi_m \in [0, t_c)$. This removes any dependence on the adaptive internal time grid and ensures pointwise comparability between the baseline and perturbed waveforms.

Observation maps and sensitivity matrix

Two observation maps are considered:

$$\mathcal{H}_A(\mathbf{x}) = [P_{ao}(\phi)] = [x_1(\phi)]\,, \tag{4}$$
$$\mathcal{H}_B(\mathbf{x}) = [P_{ao}(\phi),\, P_{pa,\mathrm{prox}}(\phi),\, V_{LV}(\phi)] = [x_1(\phi),\, x_7(\phi),\, x_{12}(\phi)]\,. \tag{5}$$

Map $\mathcal{H}_A$ represents a restricted measurement configuration, whereas $\mathcal{H}_B$ combines systemic pressure, pulmonary pressure, and ventricular volume information. For observable y_k, phase sample ϕ_m, and parameter p_j, define

$$J_{(k,m),j} = \frac{1}{s_k}\frac{y_k(\phi_m; p_j^+) - y_k(\phi_m; p_j^-)}{2\varepsilon} \approx \frac{1}{s_k}\frac{\partial y_k(\phi_m)}{\partial \ln p_j}, \tag{6}$$

where

$$s_k = \mathrm{ptp}\left[y_k^{(0)}(\phi)\right] \tag{7}$$

is the peak-to-peak amplitude of the corresponding baseline waveform. This scaling removes the incompatibility between pressure and volume units. If measurement-error standard deviations are known, they should replace Eq. (7); the resulting matrix is then weighted by the information content of the measurements rather than by waveform amplitude alone.

A compact scalar measure of the local influence of parameter p_j is

$$I_j = \frac{\|J_{:,j}\|_2}{\sqrt{N_{\mathrm{obs}}}}, \tag{8}$$

where $N_{\mathrm{obs}} = M$ for $\mathcal{H}_A$ and $N_{\mathrm{obs}} = 3M$ for $\mathcal{H}_B$. The values I_j provide a dimensionless local ranking for the selected parameter set. They do not measure global importance outside the $\pm 5\%$ neighborhood.

Singular-value analysis

For each observation map, compute

$$J = U\Sigma V^T, \qquad \Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r), \qquad \sigma_1 \geq \cdots \geq \sigma_r \geq 0. \tag{9}$$

Report the normalized spectrum $\widehat{\sigma}_i = \sigma_i/\sigma_1$ and the spectral condition number

$$\kappa_2(J) = \frac{\sigma_1}{\sigma_r}. \tag{10}$$

A large value of κ_2 indicates that at least one local parameter direction produces a much weaker observable response than another. The right singular vector $\mathbf{v}_r$ associated with σ_r identifies the least-informed linear combination of the logarithmic parameter perturbations. Its coefficients should be reported after the normalization $\max_j |v_{r,j}| = 1$.

No universal singular-value cutoff should be imposed without a noise model. The comparison between $\mathcal{H}_A$ and $\mathcal{H}_B$ is therefore based on the complete normalized spectra, the condition numbers, and the composition of their least-informed directions. The central technical question is whether adding pulmonary-pressure and ventricular-volume observations improves the conditioning of the local inverse problem.

Reproducible implementation

The complete calculation is implemented in `examples/07_sensitivity_identifiability.py`. The critical matrix construction and decomposition are

```
for j, name in enumerate(parameter_names):
    y_plus = perturbed[(name, "plus")]
    y_minus = perturbed[(name, "minus")]
    d_y_d_logp = (y_plus - y_minus) / (2.0 * epsilon)
    J_full[:, j] = (d_y_d_logp / scales[:, None]).reshape(-1)

J_A = J_full[:n_phase, :]
J_B = J_full

for label, J in {"H_A": J_A, "H_B": J_B}.items():
    _, singular_values, Vt = np.linalg.svd(J, full_matrices=False)
    relative_spectrum = singular_values / singular_values[0]
    condition_number = singular_values[0] / singular_values[-1]
    least_informed = Vt[-1] / np.max(np.abs(Vt[-1]))
    influence = np.linalg.norm(J, axis=0) / np.sqrt(J.shape[0])
```

Listing 6: Dimensionless waveform-sensitivity matrix and singular-value screening.

Run the protocol with

```
python examples/07_sensitivity_identifiability.py
```

The script records the convergence error for every perturbed case and saves the sensitivity matrices, parameter influence indices, singular spectra, condition numbers, and least-informed parameter directions in a machine-readable exttt.npz file.

R Reproducibility check — This calculation is a local practical-identifiability screening around the adopted baseline and under the two specified observation maps. It is not a proof of structural identifiability, does not produce parameter confidence intervals, and does not replace a global sensitivity analysis. Those conclusions would require, respectively, an analysis of the symbolic model-observation structure, an explicit statistical measurement model, and joint sampling over justified parameter domains.

Required technical report

The analysis should be documented through the following deliverables:

1. a convergence table containing the twelve perturbed cases and their final values of η_{cyc};
2. the local influence indices I_j for $\mathcal{H}_A$ and $\mathcal{H}_B$, with a physical interpretation of the dominant parameters;
3. the two normalized singular-value spectra and the corresponding condition numbers;
4. the coefficients of the least-informed direction $\mathbf{v}_r$ for each observation map; and
5. a concise assessment of whether the expanded observation map improves local parameter discrimination, together with an explicit statement of why the calculation does not establish structural identifiability.

9 Educational use

The `cardiosim` package is designed for possible use as a teaching module in advanced undergraduate or graduate-level courses in which physical modeling, numerical simulation, and scientific

computing are taught in an integrated way. Rather than functioning merely as a software artifact, the package may be organized as a structured pedagogical object through which students move from physical interpretation to mathematical formulation, from equations to code, and from code to computational experiments.

R Reproducibility check — The sequences below are proposed instructional designs. No controlled classroom study or formal learning assessment has yet been conducted. Accordingly, this supplement documents pedagogical potential and implementation pathways; it does not claim empirically demonstrated educational effectiveness.

A productive strategy is to organize the material in short lecture–lab sequences, each centered on a different disciplinary perspective. The computational core remains the same throughout, but the emphasis shifts according to the learning objectives of the course.

Computational Physics

T Pedagogical note — Recommended sequence.

1. **Lecture 1.** Physical analogies: RLC circuits and cardiovascular dynamics. Derivation of the twelve-state system from Kirchhoff's laws and interpretation of the state variables.
2. **Lecture 2.** Adaptive Runge–Kutta methods, tolerance control, and stiffness. Suggested pairing: Experiment 4.
3. **Lecture 3.** Numerical experimentation as method: parameter-variation studies as the numerical analogue of controlled experiments. Suggested pairing: Experiments 1 and 2.
4. **Lecture 4.** Long-time integration, periodic regimes, and attracting cycles. Suggested pairing: Experiment 6.

Pedagogical emphasis. The model is intended to show how a physically interpretable system becomes a numerical object through the construction of `rhs(t, y)` and the solver interface.

Biophysics

Pedagogical note — Recommended sequence.

1. **Lecture 1.** The cardiac cycle as a dynamical process. Baseline simulation and interpretation of pressure, flow, and volume signals.
2. **Lecture 2.** Pressure–volume loops as geometric signatures of cardiac work. Suggested pairing: baseline PV loops and Experiment 3.
3. **Lecture 3.** Pathophysiology through parameters. Coordinated laboratory session based on Experiments 1, 2, and 3.
4. **Lecture 4.** Scope and limits of lumped-parameter models: what is captured, what is simplified, and why that simplification can still be scientifically useful.

Pedagogical emphasis. The model is intended to help students examine how physiological interpretation is not detached from mathematics: the geometry of the PV loop, the timing of the cardiac cycle, and the role of resistance and elastance all emerge from the same dynamical system.

Mathematical Modeling

Pedagogical note — Recommended sequence.

1. **Lecture 1.** State-space formulation of coupled systems: the map $\Phi(\mathbf{x}, t)$ as the central abstraction.
2. **Lecture 2.** Nonlinear switching, diode models, and hybrid features in continuous-time systems. Suggested pairing: valve logic and Experiment 3.
3. **Lecture 3.** Parameter sensitivity, identifiability, and model reduction strategies. Suggested pairing: §8.
4. **Lecture 4.** Model assumptions, domains of validity, and the trade-off between fidelity, interpretability, and tractability.

Pedagogical emphasis. The package provides a setting in which to discuss not only how a model is solved, but also how it is constructed, what its assumptions mean, and how changes in parameters reorganize the global response.

Numerical Methods

Pedagogical note — Recommended sequence.

1. **Lecture 1.** From `rhs(t, y)` to `solve_ivp`: the ODE-solver interface.
2. **Lecture 2.** Stability, accuracy, and adaptivity: tolerance ladders and solver comparison. Suggested pairing: Experiment 4.
3. **Lecture 3.** Long-time integration, asymptotic behavior, and periodicity diagnostics. Suggested pairing: Experiment 6.
4. **Lecture 4.** Post-processing as part of the numerical pipeline: derived observables, cycle metrics, and pressure–volume loops.

Pedagogical emphasis. The model is intended to illustrate that numerical analysis does not end when the integrator returns a trajectory: the interpretation of the output depends on carefully defined derived quantities, cycle extraction, and visualization choices.

Suggested teaching modes

T Pedagogical note — Depending on the audience, the material may be deployed in at least three complementary modes:

1. **Equation-to-code mode.** Best suited for students seeing state-space modeling and scientific Python for the first time.
2. **Experiment-driven mode.** Best suited for laboratory classes, in which each parameter change is treated as a controlled computational experiment.
3. **Project mode.** Best suited for final assignments, in which students extend the package by adding a new parameter-variation study, a new diagnostic metric, or a new visualization.

In all three modes, the intended educational contribution is the same: the model may function as an interdisciplinary bridge linking physical intuition, mathematical formulation, and computational practice within a single coherent framework. Evaluating the effectiveness of these modes through classroom implementation remains a natural next step.

10 Repository and availability

Upon acceptance of the manuscript, the full implementation associated with this work will be released in a public, version-controlled repository. A citable archived version with a persistent DOI will also be created to ensure long-term reproducibility, accessibility, and reuse.

The public release will include the refactored `cardiosim/` package, the example scripts used to regenerate the figures presented in the article, the minimal test suite described in this supplement, and the documentation required for installation and execution.

Contributions welcome

This framework is designed as a starting point, not an endpoint. Pedagogically oriented extensions will be especially welcome once the public release is available, in particular: (i) additional case studies (e.g., pediatric parameter sets, pulmonary hypertension, exercise response); (ii) alternative ventricular activation laws (two-Hill elastance, breathing modulation); (iii) student-facing notebooks with exploratory narratives; and (iv) translations of the supplement into Portuguese and Spanish to broaden accessibility within Ibero-American physics education.